

SCHOOL OF INFORMATION TECHNOLOGY · APPLIED AI SERIES

Production NLP Engineering

Fine-Tuning at Scale with PEFT, LoRA & QLoRA

1 hr

Session length

3

Fine-tuning strategies compared

1

Registered, reproducible model

SESSION ROADMAP

What We're Covering in 60 Minutes

01

The Adaptation Problem

Why full fine-tuning doesn't scale in industry

8 min

02

PEFT & LoRA Mechanics

Low-rank adapters, and how QLoRA adds quantization

12 min

03

HF Production Ecosystem

transformers, datasets, evaluate, accelerate, PEFT, W&B

10 min

04

Live Colab Benchmark

Full FT vs LoRA vs QLoRA — memory, time, F1

15 min

05

Packaging & Versioning

HF Hub and MLflow Model Registry

8 min

06

Industry Design Challenge

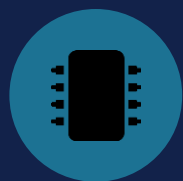
Mapping strategy to business constraints

7 min

WHY THIS MATTERS

The Adaptation Problem in Industry

You already know Transformers can be fine-tuned end-to-end. In production, that approach breaks down fast.



GPU memory explodes

Full fine-tuning stores gradients + optimizer states for every parameter — a 7B model needs ~80GB+ just to train.



One model per task doesn't scale

Support triage, contract review, clinical notes each need their own fine-tune — full copies multiply storage cost.



Iteration speed suffers

Teams need to test 10+ hyperparameter variants a week; full fine-tuning makes each run expensive and slow.

THE SOLUTION

Parameter-Efficient Fine-Tuning (PEFT)

Core idea

Freeze the pretrained backbone. Train only a small number of new or reparameterized weights that steer the model toward your task.

- Typically 0.1%–1% of parameters are trainable
- Base model weights stay frozen and shared across tasks
- Adapters are small enough to store per-task (megabytes, not gigabytes)
- Comparable accuracy to full fine-tuning on most classification tasks

PEFT family

Adapter layers

Small bottleneck MLPs inserted between Transformer blocks

LoRA

Low-rank update matrices injected into attention projections

QLoRA

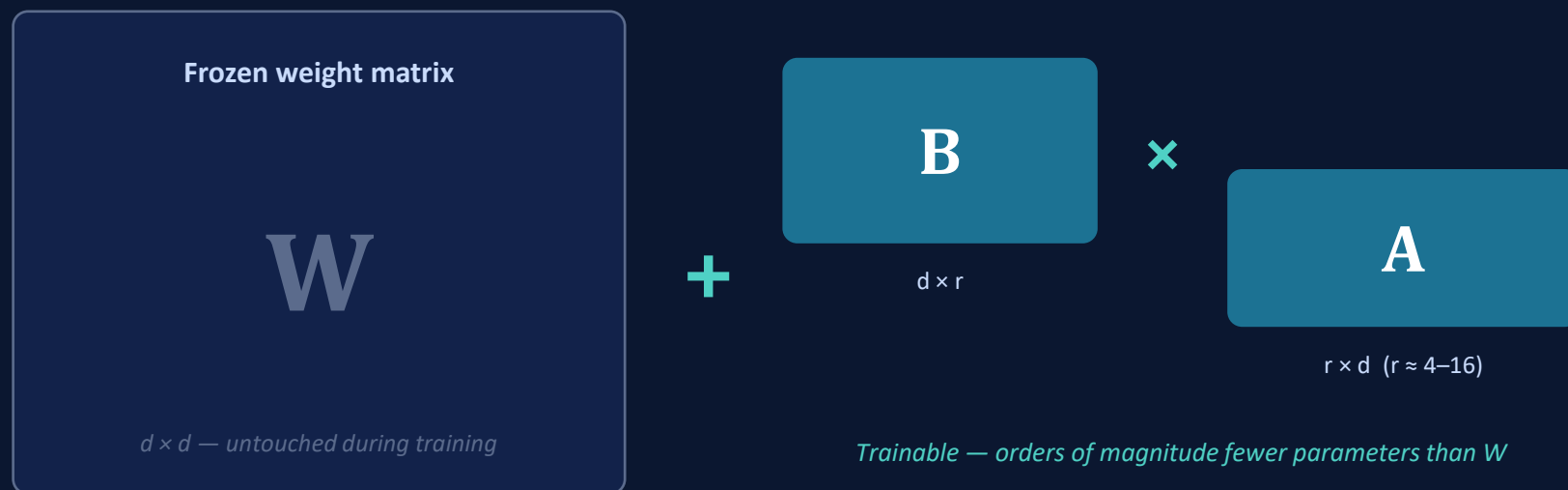
LoRA on top of a 4-bit quantized frozen backbone

Prompt / Prefix tuning

Learnable soft tokens prepended to the input — not covered today

HOW IT WORKS

LoRA: Low-Rank Adaptation



Rank r and target modules are the two hyperparameters you'll tune in the Colab notebook.

Forward pass

$$h = Wx + (\alpha/r) \cdot BAx$$

- Applied to attention Q/V (or Q/K/V/O) projection matrices
- α scales the adapter's contribution
- At inference, BA can be merged into W — zero extra latency

EXTENDING LORA

QLoRA: Quantization + LoRA

QLoRA adds one more move: load the frozen backbone in 4-bit precision, then train LoRA adapters in full precision on top.



1. Quantize

Backbone weights compressed to 4-bit NormalFloat (NF4), cutting memory ~4x vs FP16



2. Attach LoRA

Small trainable adapters (BF16/FP16) inserted at attention projections, as before



3. Compute in higher precision

De-quantize on-the-fly for matmuls; gradients only ever touch the LoRA adapters

When does the extra complexity pay off?

- Model is large enough that FP16 weights alone strain your GPU (7B+ parameters)
- You need to fit training on a single consumer or free-tier GPU
- For small BERT-family encoders (our Colab demo), memory savings are modest — but the workflow transfers directly to larger models

TOOLING

The HuggingFace Production Ecosystem



transformers

Model architectures, tokenizers, and the Trainer API



datasets

Streaming-friendly loading, mapping, and caching for large corpora



evaluate

Standardized metrics (F1, accuracy, ROUGE) shared across the community



accelerate

Hardware-agnostic multi-GPU, mixed-precision, and distributed training



peft

LoRA, QLoRA, and other adapter methods on top of any transformers model



W&B

Experiment tracking — losses, metrics, GPU usage, hyperparameters

SCALING TRAINING

Trainer API + Accelerate

HuggingFace Trainer

- Handles the training loop: forward, backward, optimizer step, logging
- `TrainingArguments` configures epochs, batch size, learning rate schedule, precision
- `report_to="wandb"` streams metrics automatically — no manual logging code
- Works identically whether you're doing full FT or PEFT — just swap the model

Accelerate

- Abstracts away device placement — same code runs on CPU, 1 GPU, or many
- Mixed precision (fp16/bf16) cuts memory and speeds up matmuls
- Trainer uses Accelerate under the hood; direct use gives you a custom loop when needed
- On free Colab (single T4), Accelerate mainly buys you mixed-precision training

EXPERIMENT DISCIPLINE

Tracking Every Run with Weights & Biases

```
import wandb

wandb.init(project="nlp-peft-benchmark", name="lora-r8")

training_args = TrainingArguments(
    report_to="wandb",
    run_name="lora-r8",
    ...
)
```

Every experiment logged this way is reproducible: hyperparameters, hardware, git commit, and metrics all live in one place.

What we log per run

- Training / eval loss curves
- F1, precision, recall per epoch
- GPU memory allocated (logged manually via torch.cuda)
- Wall-clock training time
- Full hyperparameter config

LIVE COLAB BENCHMARK

Full Fine-Tuning vs LoRA vs QLoRA

Strategy	Trainable Params	Peak GPU Memory	Relative Train Time	Eval F1
Full Fine-Tuning	~110M (100%)	Highest	Baseline (1.0x)	Reference
LoRA (r=8)	~0.3M (<1%)	~40–60% lower	Comparable or faster	Within ~1–2 pts of full FT
QLoRA (4-bit + r=8)	~0.3M (<1%)	Lowest per-parameter footprint	Slightly slower per step (dequant overhead)	Comparable to LoRA

Exact numbers depend on your dataset subset and hyperparameters — students fill in this table live from their own Colab run.

- LoRA reaches near-parity F1 while touching under 1% of parameters
- QLoRA's memory advantage grows with model size — modest here, dramatic at 7B+
- For a small BERT-family model, LoRA alone is often the pragmatic production choice

MODEL VERSIONING DISCIPLINE

Packaging & Registering Your Model



HuggingFace Hub

- `model.push_to_hub()` uploads weights, config, and tokenizer
- Merge LoRA adapters first, or push adapter-only for a lightweight artifact
- Model card documents dataset, metrics, and intended use
- Versioned automatically via git-based commits on the Hub



MLflow Model Registry

- `mlflow.transformers.log_model()` captures the pipeline + environment
- Registry tracks stages: Staging → Production → Archived
- Ties a model version back to the exact run's params and metrics
- Common choice when the org already standardizes on MLflow for MLOps

FROM NOTEBOOK TO PRODUCTION

Industry Deployment Patterns



Batch Inference

Score large volumes on a schedule — nightly contract backlogs, end-of-day ticket summaries. Optimizes for throughput, not latency.



Streaming APIs

Real-time endpoint behind a REST/gRPC service — live chat triage, in-the-moment clinical note tagging. Optimizes for low p99 latency.



Serverless Endpoints

Auto-scaling inference (e.g., HF Inference Endpoints, SageMaker Serverless) — spiky, unpredictable traffic without idle GPU cost.

Fine-tuning strategy influences deployment: LoRA adapters merge into the base model for zero-overhead serving, or stay separate to hot-swap between tasks on one shared backbone.

APPLY IT

Design Challenge: Match Strategy to Business Constraint



Support Ticket Triage

CONSTRAINT

High volume, low latency budget, frequent new intents

A DEFENSIBLE STRATEGY

LoRA on a small encoder — cheap to retrain weekly as new intents appear



Contract Analysis

CONSTRAINT

Long documents, high accuracy bar, infrequent retraining

A DEFENSIBLE STRATEGY

Full fine-tune or larger LoRA rank — accuracy matters more than update speed



Clinical Notes Classification

CONSTRAINT

Sensitive data, on-prem GPU constraints, compliance review

A DEFENSIBLE STRATEGY

QLoRA — fit training on limited on-prem hardware while meeting data-residency rules

KEY TAKEAWAYS

What You Leave With Today



A working mental model of LoRA and QLoRA

and why they dominate real-world LLM adaptation



A reproducible, benchmarked fine-tuning run

full FT vs LoRA vs QLoRA, tracked end-to-end in W&B



A registered model artifact

on HuggingFace Hub or MLflow, ready to hand to an inference team



A framework for choosing a strategy

given GPU budget, latency needs, and compliance constraints

Next: open the Colab notebook — sections map 1:1 to slides 5, 6, 9, 10 and 11.